ADAPTIVE RESOURCE QUARANTINE

Invention Disclosure Format IDF B

Project team: Rithika.S (24BCE2984)

K. Vaishnavi (24BCT0183)

Vellore Institute of Technology

Operating Systems Lab | BCSE303P

30 October 2026

© VIT IPR&TTCELL

VIT® Vellore Institute of Technology (Deemed to be University under section 3 of UGC Act, 1956) **Invention Disclosure Format (IDF)-B**	Document No.	02-IPR-R003
	Issue No/Date	2/01.02.2024
	Amd. No/Date	0/00.00.0000

1. Title of the invention:

ARQ: Bracketed Verification of CPU-Quarantine Relaxation Using Grant-Normalized Consumption and Protected-Service Stability Checks

2. Field / Area of invention:

Computer Science and Engineering: operating-system resource control, process scheduling, performance isolation and monitored restoration of restricted execution.

3. Prior Patents and Publications from literature (provide a table summarizing the prior art)

Patent or publication	Reference and year	Technical relevance
I. Segregating suspicious threads to prevent CPU resource exhaustion	US7530072B1; U.S., 2009	Quarantine, CPU/priority restriction and exit criteria are established. Broad quarantine-and-recovery claims overlap.
II. Automatic isolation of misbehaving processes	US9280662B2; U.S., 2016	Keeps misbehaving processes operating in isolation for remediation.
III. Dynamic resource allocation for tail-latency SLOs	US20230128059A1; U.S., 2023	Resource adjustment for latency-sensitive applications is existing subject matter.
IV. Performance Anomaly Detection and Bottleneck Identification	Ibidunmoye, Hernandez-Rodriguez and Elmroth (2015). ACM Computing Surveys. [1]	Surveys performance anomaly detection and bottleneck analysis. Supports telemetry analysis; does not validate ARQ thresholds.
V. Isolation Forest	Liu, Ting and Zhou (2008). ICDM. [2]	Uses isolation trees to score unusual observations. Standard model baseline; ARQ evaluates it and a support-aware candidate separately from bracketed recovery.
VI. Long Short-Term Memory	Hochreiter and Schmidhuber (1997). Neural Computation. [3]	Introduces recurrent modeling of temporal dependencies. Background only; no trained LSTM is established here.
VII. LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection.	Malhotra et al. (2016). [4]	Detects unusual sequences through reconstruction error. Sequence reconstruction is prior work, not a new ARQ mechanism.
VIII. Improving Resource Efficiency at Scale with Heracles	Lo et al. (2016). ACM TOCS. [5]	Manages colocated resources with feedback to protect latency. Latency-driven allocation and recovery are established.

Patent or publication	Reference and year	Technical relevance
IX. Caladan: Mitigating Interference at Microsecond Timescales	Fried et al. (2020). OSDI. [6]	Uses fast allocation and contention signals for isolation. ARQ does not reproduce its scheduler or timing scale.
X. PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services	Chen, Delimitrou and Martinez (2019). ASPLOS. [7]	Dynamically partitions resources for colocated interactive services. Multi-service QoS feedback is not novel to this project.
XI. Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads	Ousterhout et al. (2019). NSDI. [8]	Adjusts CPU allocations to changing service demands. Provides a scheduling comparison, not an ARQ implementation.
XII. CPI2: CPU performance isolation for shared compute clusters	Zhang et al. (2013). EuroSys. [9]	Uses CPI observations to identify interference and throttle likely offenders. Interference-aware selective throttling already exists.
XIII. Quasar: Resource-Efficient and QoS-Aware Cluster Management	Delimitrou and Kozyrakis (2014). ASPLOS. [10]	Combines classification with resource allocation and assignment. Learned resource decisions are established research.
XIV. Paragon: QoS-aware scheduling for heterogeneous datacenters	Delimitrou and Kozyrakis (2013). ASPLOS. [11]	Accounts for hardware heterogeneity and workload interference. A broader placement problem than local CPU-quarantine release.
XV. Bubble-Up: Increasing Utilization In Modern Warehouse Scale Computers Via Sensible Co-Locations	Mars et al. (2011). MICRO. [12]	Applies tunable memory pressure to characterize interference sensitivity. Active perturbation for interference assessment is known.
XVI. Resource Containers: A New Facility for Resource Management in Server Systems	Banga, Druschel and Mogul (1999). OSDI. [13]	Separates resource accounting principals from protection domains. Resource management and security isolation are distinct.
XVII. Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center	Hindman et al. (2011). NSDI. [14]	Coordinates fine-grained sharing across cluster frameworks. Cluster allocation differs from this local recovery experiment.

Patent or publication	Reference and year	Technical relevance
XVIII. Large-scale cluster management at Google with Borg	Verma et al. (2015). EuroSys. [15]	Describes large-scale workload placement and cluster management. Deployment scale and guarantees are not comparable to ARQ.
XIX. Omega: flexible, scalable schedulers for large compute clusters	Schwarzkopf et al. (2013). EuroSys. [16]	Uses shared state and optimistic concurrency for scheduling. Does not establish this prototype as a cluster scheduler.
XX. Dominant Resource Fairness: Fair Allocation of Multiple Resource Types	Ghodsi et al. (2011). NSDI. [17]	Defines multi-resource fair allocation. ARQ does not implement DRF or prove allocation fairness.
XXI. Autopilot: Workload Autoscaling at Google Scale	Rzadca et al. (2020). EuroSys. [18]	Automates resource sizing and scaling for workloads. Autoscaling evidence cannot be transferred to ARQ.
XXII. Pliant: Leveraging Approximation to Improve Datacenter Resource Efficiency	Kulkarni et al. (2019). HPCA. [19]	Uses approximation to reduce colocation contention. ARQ changes CPU duty, not application approximation quality.
XXIII. Dune: Safe User-level Access to Privileged CPU Features	Belay et al. (2012). OSDI. [20]	Exposes privileged CPU facilities safely to a Linux process using virtualization hardware. ARQ uses ordinary process controls; it does not implement Dune or hardware-backed isolation.
XXIV. Shinjuku: Preemptive Scheduling for Microsecond-scale Tail Latency	Kaffes et al. (2019). NSDI. [21]	Uses fine-grained preemption to reduce head-of-line blocking across variable-length requests. Scheduling latency is established prior work. ARQ has no comparable microsecond scheduler.
XXV. Overload Control for Microsecond-scale RPCs with Breakwater	Cho et al. (2020). OSDI. [22]	Uses queue-delay-driven admission credits, demand speculation and overload handling. Increasing a grant to expose unmet demand has conceptual overlap; admission credits differ from CPU-duty relaxation.

Patent or publication	Reference and year	Technical relevance
XXVI. FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices	Qiu et al. (2020). OSDI. [23]	Uses telemetry and machine learning to localize contention and adapt resource provisioning. ML-assisted SLO control is established. ARQ must justify its particular recovery protocol rather than claim ML control generally.
XXVII. Sinan: Data-Driven, QoS-Aware Cluster Management for Microservices	Zhang et al. (2021). arXiv:2105.13424. [24]	Models microservice dependencies to allocate resources while protecting end-to-end tail latency. ARQ does not model a microservice dependency graph or validate comparable end-to-end QoS.
XXVIII. Sage: Leveraging ML to Diagnose Unpredictable Performance in Cloud Microservices	Gan et al. (2021). arXiv:2112.06263. [25]	Uses unsupervised models for online root-cause diagnosis and corrective performance actions. Root-cause inference and correction are known; ARQ gate outcomes do not identify a unique cause.
XXIX. cpulimit	Existing open-source software; additional source	SIGSTOP/SIGCONT CPU duty control is established; the actuator is not claimed as new.

Table I: Prior patents and 25 research publications related to resource isolation, anomaly detection, latency-aware allocation and recovery verification. Roman numerals identify table rows; bracketed numbers identify the linked research references. Patents and software do not count toward the 25-paper minimum.

The proposed distinction is the precise recovery-verification protocol below, not the broad idea of monitoring, throttling, probing or feedback control. Novelty and inventive step remain unresolved and require a full professional search.

3.1 Synthesis and precise candidate distinction

Detection literature [1-4] separates unusual observations from diagnosis. An anomalous reading is not proof of harmful behavior, and a normal reading under throttling is not proof of recovery. The current small evaluation measures classification error; it does not validate restoration decisions.

Latency-aware allocation [5-12], resource accounting and cluster management [13-18], and approximation-based control [19] show that monitoring, allocation changes and protected-service objectives are mature ideas. Scheduler designs [20-21] also demonstrate why control timing and actuator semantics matter.

Breakwater [22] is particularly relevant because speculative additional credits can reveal unmet demand. FIRM, Sinan and Sage [23-25] establish telemetry-driven resource correction and learned diagnosis. Combining probing, ML and a latency threshold is therefore not a sufficient novelty argument.

Candidate detail	**Evidence and remaining question**
Repeated A-B-A windows	Implemented in recovery_probe.py; compare exact protocols and claim language with full prior-art disclosures.
Grant-normalized CPU consumption	Implemented demand gate. Blocking, contention and multicore behavior can still confound interpretation.
Restoration before decision	Implemented cleanup ordering; ordinary rollback and defensive cleanup are established techniques.
Combined stability contract	Identity, validity, load, drift and latency gates are implemented. Whether this conjunction is inventive remains unresolved.

The defensible research question is whether this particular contract reduces premature release relative to the legacy recovery rule under changing demand. A representative live experiment and ablations are needed; no claim of superiority over the 25 published systems is made.

4. Summary of the invention:

ARQ is an operating-systems prototype that observes process telemetry and applies graduated execution restrictions. Its original recovery rule accepts repeated low anomaly readings as evidence for stepping down restrictions. That can be misleading: the restriction itself may have reduced the observed CPU usage while the process still wants more execution time.

The research extension tests one proposed increase from a restricted allocation A to a larger allocation B. Each release window B is bracketed by restricted control windows A. The runner checks measurement validity, process incarnation, offered-load stability, control-window latency drift, protected-service latency and CPU consumption relative to the larger grant. It reapplies A before committing a successful result. A failed or inconclusive experiment retains the restriction.

Technical problem and effect

The technical problem is premature relaxation based on controller-suppressed observations. The intended effect is more selective CPU-duty relaxation using additional experimental evidence. Regression tests and an explicit synthetic model support the implemented logic. Live benefits of this bracketed policy, production safety and generalization are not established.

Scope and current implementation

The bracketed runner and throttler adapter are implemented as an opt-in research API. They are separate from default dashboard restoration. Supporting work includes process-level detector evaluation, a support-aware candidate and guarded optional ML assistance. A new Kaggle study trains server-level Isolation Forest and Extra Trees models on 94,526 cleaned unique observations. Its collection domains are confounded with labels, and its features cannot replace the live process model. These studies do not establish the novelty of the bracketed protocol.

This document is a technical disclosure draft. It is not a filed application or a patent grant. Team membership does not establish legal inventorship; actual human conception, AI assistance and institutional ownership require confirmation.

5. Objectives of the invention:

1. Verify one proposed relaxation of an existing CPU restriction using repeated A-B-A observation groups.

2. Distinguish low restricted usage from continued saturation of a larger temporary grant.

3. Check the protected workload against absolute and relative p95 latency bounds.

4. Treat offered-load changes, control drift and invalid observations as inconclusive rather than recovered.

5. Reserve a nominal incremental execution allowance before actuation and restore the original restriction before deciding.

6. Retain phase observations, outcomes and rejection reasons in a trace.

7. Provide reproducible tests and labelled evaluation while keeping unvalidated ML out of automatic control.

Measurable outcomes

Observe whether a candidate is accepted or rejected, the reason, final requested duty and recorded nominal allowance. For detector evaluation, report confusion counts, precision, recall, F1, false-positive/negative rates and agreement with rule decisions. For repeated live trials, report measured processing-time percentiles and throughput with sample variability.

A lower score, fewer flags or one passing experiment is not evidence of global optimality or general recovery. Application data integrity, multicore demand, memory isolation and unobserved workload phases need separate testing.

6. Components and Working Principle:

Module	Function	Source location
Process monitor	Native telemetry and process-incarnation cache reset.	arq/core/monitor.py
Rule scorer / restoration	Resource scoring and original hysteresis transitions.	arq/core/
Duty actuator	Stop/continue pulsing with cleanup.	quarantine_controller.py
RecoveryProbe	Bracketed experiment and acceptance gates.	recovery_probe.py
Window / ProbePolicy	Observation contract and configured limits.	recovery_probe.py
Throttler adapter	Change an already restricted duty allocation.	recovery_probe.py
Guarded ML path	Validated soft-restriction corroboration; managed-child and dry-run controls.	ml_policy.py; guarded_controller.py
Evaluation tools	Labelled data, repeated fits and randomized-order live trials.	arq/evaluation/

Working principle

Check the nominal allowance; apply A and observe; apply B and observe; return to A and observe. Check the complete bracket. Repeat for the configured number of rounds. Always reapply A when leaving the experiment. If every required bracket passes, apply B as the final allocation; otherwise retain A. Failures to actuate or restore must not be reported as successful recovery.

7. Description of the invention in detail

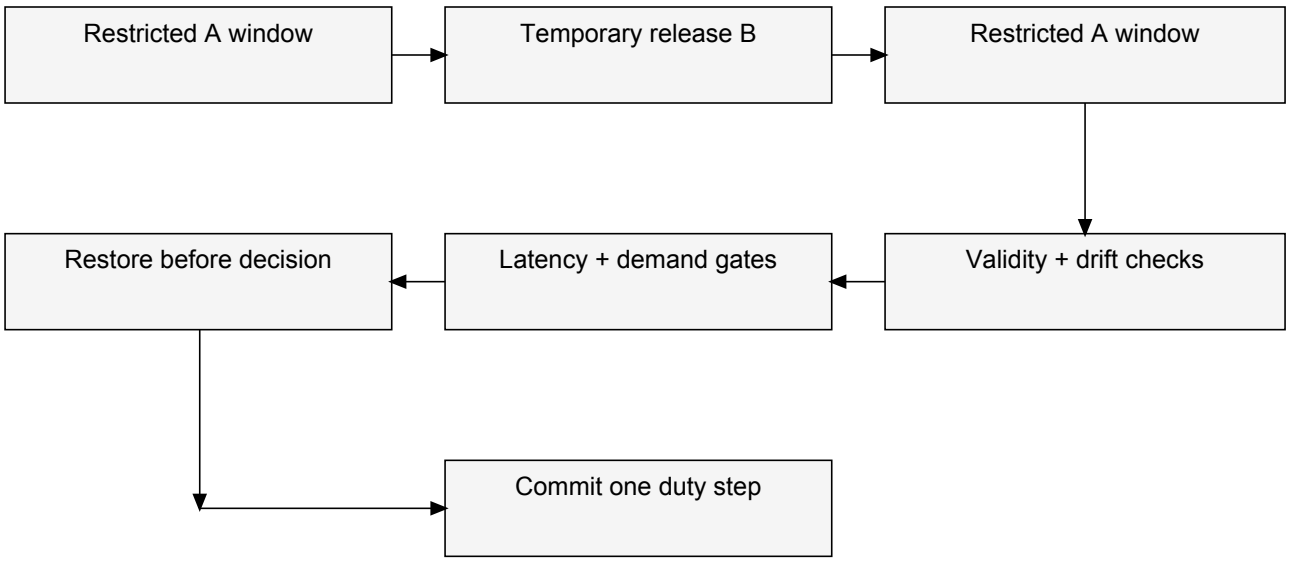


Figure 1. Opt-in bracketed verification before one duty relaxation.

7.1 Observation contract

A Window contains a process-incarnation identity, reported effective duty, duration, CPU seconds, protected-service latency samples, offered request rate and health status. It must contain finite values and sufficient observations. The prototype trusts the supplied sampler; a requested duty ratio is not a substitute for measured actuation.

7.2 Acceptance gates

All windows must meet the absolute SLO. Offered load must remain stable and the two A-window p95 values must remain close. The B-window p95 increase over the median A-window p95 must remain within its allowance. B CPU seconds divided by B duration times B duty must remain below the demand threshold. A rejected experiment does not prove the process is faulty.

7.3 Research configuration

Parameter	Default
A / B duty in retained simulation	0.15 / 0.40
Window / rounds	0.5 seconds / two complete brackets
Nominal additional grant budget	0.30 one-core seconds per call
Protected p95 / permitted increase	20 ms / 2 ms
A-window p95 drift / offered-rate drift	1 ms / 10%
Maximum grant-normalized consumption	0.70
Minimum latency samples per window	20

7.4 Algorithm

```
reserve = (B - A) * window_seconds * rounds
if reserve > budget: reject before actuation
try:
    for each required round:
        before = apply_and_observe(A)
        trial  = apply_and_observe(B)
        after  = apply_and_observe(A)
        check validity, load stability and control drift
        check protected latency and normalized demand
        if any gate fails: retain restriction
finally:
    apply(A)
if every complete bracket passed:
    try: apply(B)
    on failure: apply(A); raise
return decision, final requested duty, phase trace
```

7.5 Supporting validation and guarded control

The current project supports disjoint train, calibration and test runs. Training/calibration contain normal observations; test data contains both labels. It reports detector errors and agreement, repeats model fits with fixed seeds, and runs randomized-order repeated live benchmark trials. A candidate detector supplements Isolation Forest with a robust normal-training support envelope.

Optional assisted control requires a model hash matching a recent evaluation report, minimum held-out counts and configured quality bounds. ML must corroborate a rule watchlist score for three observations and can only elevate it to soft restriction. It cannot independently authorize restoration or termination. The supplied small dataset fails the deployment gate.

The dashboard defaults to dry-run and loopback binding. Enabled actuation is limited to registered live children, priority changes are disabled in that guarded path, and automatic termination is separately opt-in. An in-process heartbeat lease releases restrictions after expiry. These are supporting implementation features, not a claim that ML and the bracketed runner have been integrated into one controller.

7.6 Limits

The nominal grant budget is not a hard real-time bound. An unresponsive sampler, daemon death, failed signal delivery or PID races require stronger external supervision and OS facilities. Bracketing does not prove causality; low grant-normalized consumption may reflect blocking or contention. Cross-platform equivalence, generalization and application-state preservation remain unproven.

7.7 Prepare the screenshot environment

Run commands from the project root. The complete project ZIP includes the retained controlled JSONL and evaluated models. A source-only clone requires collecting the labelled file before running SS3; the results will vary and must be shown as produced.

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements-evaluation.txt
# Only if controlled-labels.jsonl is absent:
PYTHONPATH=. .venv/bin/python scripts/collect_labelled.py
```

7.8 Quantities, units and a worked example

Quantity	Definition / interpretation
Restricted / candidate duty	A and B are requested execution fractions, with 0 < A < B <= 1 in the nominal one-core model.
Normalized trial consumption	u_B = CPU_seconds_B / (duration_B * effective_duty_B). This is a ratio, not a probability of recovery.
Additional grant reservation	E = (B - A) * window_seconds * rounds. Units: nominal one-core seconds.
Relative latency penalty	p95(B) minus median(p95(A_before), p95(A_after)).
Control drift	Absolute difference between the two restricted-window p95 values.
Precision / recall	TP/(TP+FP) and TP/(TP+FN); report undefined denominators explicitly.
FPR / F1	FP/(FP+TN) and 2TP/(2TP+FP+FN).

For A = 0.15, B = 0.40, 0.5-second windows and two rounds, E = 0.25 nominal core-seconds, below the configured 0.30 allowance. A hypothetical B window using 0.10 CPU-seconds has u_B = 0.50. This passes the 0.70 demand threshold only; every other gate must still pass. These example values are explanatory, not measurements.

Returning to A means reapplying the restriction; it does not restore application memory or rewind execution. The document therefore does not report a snapshot rollback time. Reapplication and confirmed actuator latency need explicit timestamps and a dedicated experiment before numerical claims are made.

7.9 Failure handling, integration and technical boundaries

Condition	Required behavior / limitation
PID incarnation changes	Reject the observation group; never combine telemetry from different process lifetimes.
Missing or non-finite observation	Reject as inconclusive; absence of telemetry is not a healthy zero.
Load or control drift exceeds bound	Retain the restriction; do not infer recovery from a changing experiment.
Protected latency fails a bound	Reject the candidate step even if its normalized CPU demand is low.
Sampler or actuation raises	Attempt base reapplication through finally; propagate failure instead of claiming successful recovery.
Final relaxation fails	Attempt to reapply A and expose the failure. A second failed actuation requires external intervention.
Monitor heartbeat expires	Guarded dashboard path releases managed restrictions; this in-process lease is separate from the research probe.
Daemon killed or machine fails	No independent watchdog guarantee is implemented. OS-backed supervision is future work.

The integration boundary matters: bracketed verification is an opt-in API, while dashboard restoration follows the base controller. ML assistance is separately guarded and cannot authorize recovery. An end-to-end policy integrating these paths has not been validated.

Python, native OS telemetry, process signals, standard-library HTTP serving and optional scikit-learn/joblib form the current stack. The prototype is a local research tool, not a replacement kernel scheduler, container security boundary or distributed cluster manager.

8. Experimental validation results:

8.1 Retained result summary

Evidence	Retained result / boundary
Full regression suite	61 test methods pass: the earlier 57 plus four Kaggle data-validation and deployment-boundary tests.
Native macOS repair	rusage v2 request matches its buffer; CPU accounting checked against a child process clock.
Controlled labelled collection	40 training, 40 calibration and 100 test observations; controlled scenarios, not representative deployment data.
Plain Isolation Forest	TP 0, FP 2, TN 38, FN 60 on the retained held-out collection.
Support-aware candidate	TP 57, FP 2, TN 38, FN 3; exploratory comparison on the same held-out collection.
Deployment gate	Both supplied candidates rejected; the dataset is below the minimum held-out size.
Repeated live benchmark	Nine local trials: three strategies and three one-second repeats.
Bracketed recovery experiment	500 constructed trials; results are synthetic, not live policy performance.

The support-aware candidate has 0.966 precision and 0.95 recall on this small controlled test set. These figures must not be presented as general accuracy. Model-fit repeats use the same test runs, and candidate comparison is exploratory. New untouched, representative workloads are required before deployment.

The live latency probe measures processing duration of synthetic work units, not end-to-end service latency. Application state-loss and recovery-time percentages were not measured. Linux parser tests and CI configuration are present; remote Linux/macOS CI execution is not claimed.

8.2 Output evidence - SS1

Dashboard: dry-run mode and native telemetry

SS1 - Dashboard: dry-run mode and native telemetry

Paste the actual output screenshot here

Figure 2. Current dashboard showing execution mode, telemetry backend and monitored process status.

What screenshot to paste

Run the dashboard, open the displayed localhost URL, and capture the top status cards plus process table. Show Dry run and the backend status. If a trained model is unavailable, retain that message rather than adding a simulated ML value.

```
.venv/bin/python -m arq.cli web
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.3 Output evidence - SS2

Complete regression test summary



Figure 3. Full regression-suite output from the environment used for the submission.

What screenshot to paste

Capture the command and final lines showing the number of tests, elapsed time and OK. The updated version passes 61 methods; if your version differs, show the actual count. Do not hide failures or skipped tests.

```
.venv/bin/python -m unittest discover -s tests -v
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.4 Output evidence - SS3

Held-out detector evaluation



Figure 4. Support-aware candidate evaluation on disjoint controlled runs; dataset size and error counts visible.

What screenshot to paste

Capture the candidate confusion counts and precision, recall, F1 and false-positive rate from evaluation.json. Include the partition counts or a second visible terminal block. Label it controlled-scenario evaluation; do not call it production accuracy.

```
.venv/bin/python -m arq.cli evaluate-labelled \
  --data output/validation/controlled-labels.jsonl \
  --output-dir output/validation/support-candidate \
  --repeats 3 --detector support_if
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.5 Output evidence - SS4

Unqualified model blocked from assistance

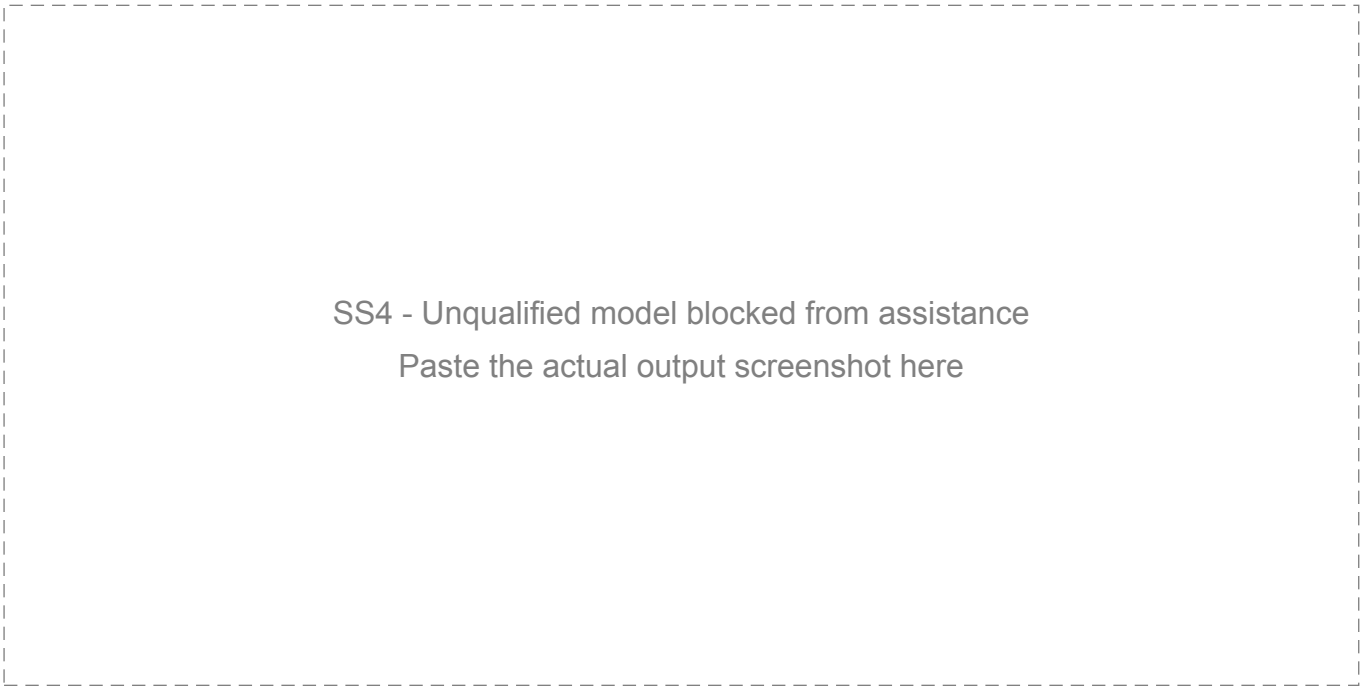


Figure 5. Assistance startup rejected because the supplied candidate lacks sufficient qualifying evaluation evidence.

What screenshot to paste

Capture the command and validation refusal for the supplied small dataset. It should state that the candidate does not meet assistance acceptance criteria. This demonstrates a blocked deployment, not a successful ML-enforcement run. Do not edit the report to bypass the gate.

```
.venv/bin/python -m arq.cli web --ml-mode assist \
  --ml-model output/validation/support-candidate/evaluated_model.joblib \
  --ml-validation output/validation/support-candidate/evaluation.json
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.6 Output evidence - SS5

Repeated live benchmark summary

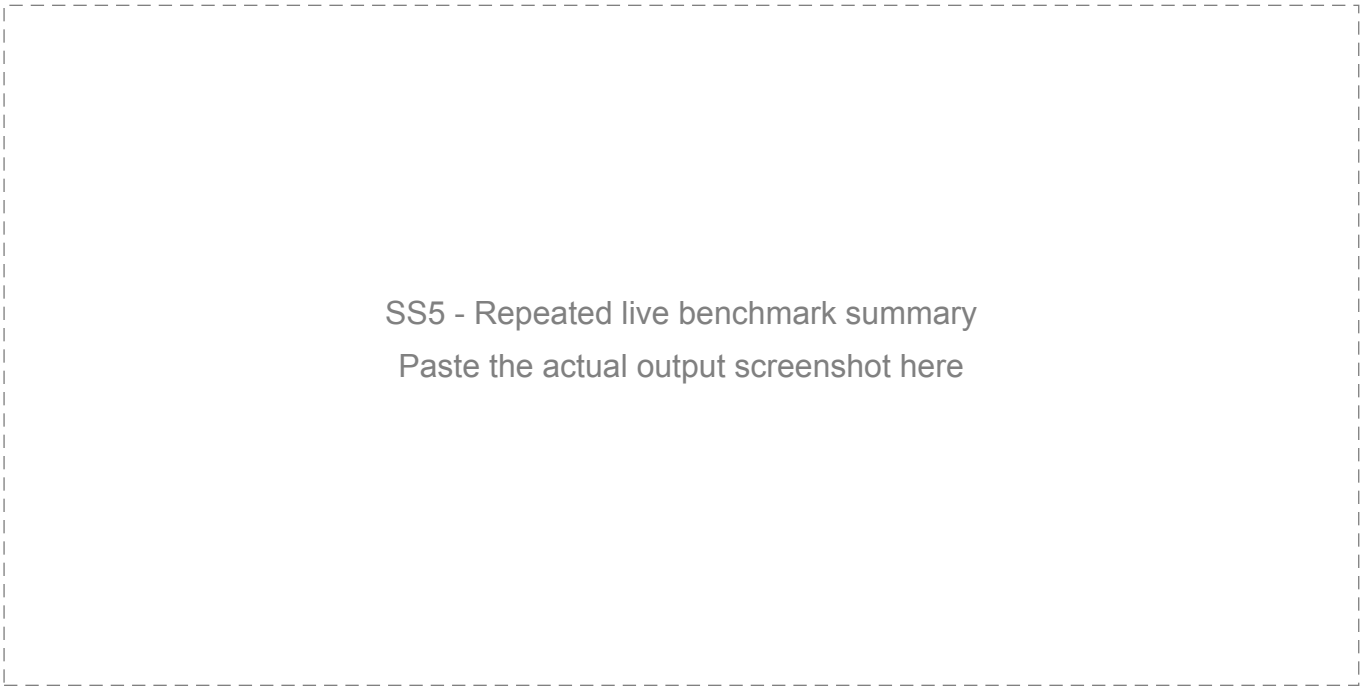


Figure 6. Repeated local live-workload trials; synthetic work-unit processing latency, not end-to-end service latency.

What screenshot to paste

Run the repeated benchmark and capture the summary means and sample standard deviations for P95, P99 and throughput. Keep the duration, repeat count and platform visible in the command or a nearby output block. Do not paste the older fixed 68.4% or 70% gain claims.

```
.venv/bin/python -m arq.cli benchmark-repeat \
  --repeats 3 --duration 1
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.7 Output evidence - SS6

Bracketed recovery comparison: synthetic experiment



Figure 7. Synthetic comparison of the legacy recovery rule and the bracketed verification policy.

What screenshot to paste

Capture the five scenario summaries with legacy_relaxations and probe_relaxations. Include a visible Synthetic model label. This validates modeled decision behavior only; it is not a screenshot of live recovery or proof of general accuracy.

```
echo "Synthetic model - not live telemetry"
PYTHONPATH=. .venv/bin/python research/recovery_probe/evaluate.py
```

Capture real output from your project environment. Preserve the command, result labels and relevant limitations. Crop unrelated windows or personal information; do not alter result values.

8.8 Reproducibility and interpretation

The retained collection contains 180 observations: 40 normal training, 40 normal calibration and 100 held-out test observations, comprising 40 normal and 60 anomalous labels. Splits are run-disjoint. Controlled labels describe workload scenarios, not verified maliciousness or every deployment failure mode.

Detector	TP / FP / TN / FN	Precision / recall / F1 / FPR
Plain Isolation Forest	0 / 2 / 38 / 60	0.000 / 0.000 / 0.000 / 0.050
Support-aware candidate	57 / 2 / 38 / 3	0.966 / 0.950 / 0.958 / 0.050

Three repeated fits reuse the same test collection. They describe sensitivity to fit randomness, not three independent workload studies. The candidate comparison is exploratory; a fresh untouched test collection is needed to measure generalization. Both candidate reports fail the minimum deployment sample counts.

The retained live benchmark has nine short local trials, three strategies and three repeats. Report raw trial values, trial order, platform, workload duration and sample standard deviation. Processing-time percentiles of synthetic work units do not establish an end-to-end service SLO, state preservation or production safety.

Synthetic recovery experiment

The 500 constructed trials comprise five modeled categories with 100 trials each. The bracketed policy accepts modeled recovery and rejects modeled sustained demand, protected-service harm, control drift and offered-load change. These outcomes are conditional on the generator assumptions; they are not measured field error rates.

8.8.1 Retained live benchmark values

Each cell reports the mean / sample standard deviation across three trials. Each latency value is a trial-level percentile; these are not percentiles of pooled request samples. Units are milliseconds for latency and requests per second for throughput.

Strategy	P95 ms	P99 ms	Throughput / s
Unprotected	0.091 / 0.001	0.207 / 0.194	39.1 / 0.173
Static termination	0.381 / 0.006	0.425 / 0.031	39.3 / 0.1
Adaptive quarantine	0.114 / 0.005	0.142 / 0.018	39.033 / 0.252

Environment: macOS-26.5.2-arm64-arm-64bit; Python 3.12.14. Configured workload duration: 1.0 second per trial; three strategies and three repeats; randomized strategy order with seed 42.

Adaptive quarantine had a higher mean P95 than the unprotected baseline in this retained run (0.114 versus 0.091 ms). Its lower mean P99 is accompanied by a variable unprotected P99. These short trials do not justify a general speedup, causal performance benefit or optimality claim. Throughput means were similar.

The controlled noisy worker survived all three adaptive and all three unprotected trials; static termination ended it in all three trials. Worker survival is not an application-state-integrity or task-completion measurement. This benchmark exercises the legacy policy, not the bracketed recovery runner.

Reproducible record: output/validation/repeated-benchmark.json. Retain the raw nine trial records when reproducing this table. New machine measurements should replace these values only with their actual corresponding records.

Evidence boundary

No published comparison system in Section 3 was executed here. Literature comparison is qualitative; local baseline comparison is empirical but short. The synthetic recovery study is a separate modeled experiment. Combining their numbers into a single headline accuracy or speedup would be misleading.

8.9 Kaggle dataset: source, audit and partitions

A larger external study uses the Cryptojacking Attack Timeseries Dataset by Keshani Jayasinghe, published on Kaggle with a CC0 data-card licence. The normal and anomalous CSV file names supply the binary labels. These labels have not been independently verified for every observation. The combined CSV is excluded to avoid reusing the same collection.

Kaggle dataset source and publisher documentation

Preparation step	Observed count / procedure
Raw source records	80,851 normal plus 14,461 anomalous = 95,312 records.
Invalid records	Two rejected for invalid numeric values or timestamps.
Duplicate feature vectors	784 removed before partitioning; no contradictory feature vectors retained.
Clean unique observations	94,526 records.
Boundary purge	2,005 records excluded around chronological split boundaries.
Training partition	56,170: 47,649 normal and 8,521 attack-file records.
Calibration partition	17,922: 15,217 normal and 2,705 attack-file records.
Test partition	18,429: 15,619 normal and 2,810 attack-file records.

Each host is divided chronologically into nominal 60/20/20 partitions. Equal timestamps stay together. One hundred unique timestamps on each side of each boundary are purged. Exact selected-feature duplicates are removed before this split. The manifest retains source file, line, host, timestamp, label and assigned partition.

Archive SHA-256: fb6277bb1ea73fc431a6162576b8765e68a38a528d9a55ac97672a4867f3069c. Hashes and provenance are retained in evaluation.json and the downloadable training package. The archive contains 82 columns per CSV, not 246 independent telemetry features; the Kaggle explorer totals the columns of three files.

8.10 Kaggle training methods and measured results

The study selects 23 numeric host-resource features covering CPU state percentages, load averages, memory gauges and process counts. One constant training feature is removed, leaving 22 model inputs. Hostnames, dates, OS strings, source labels, static hardware identity and cumulative counters are excluded. These are whole-server readings, not process RSS, process VMS or process-specific I/O rates.

Model	Training and threshold
Isolation Forest	200 trees; max_samples 2,048; trained only on 47,649 normal training observations. Threshold: 95th percentile of negative decision scores on normal calibration observations.
Extra Trees	200 trees; minimum leaf size 5; square-root feature sampling; balanced class weights. Trained on all 56,170 training rows. Fixed positive-class threshold 0.5.

Three fits use seeds 42, 43 and 44. The table reports mean metrics over the same held-out set, not independent workload collections. Test labels are used only to compute metrics; no test-set threshold tuning is performed.

Model	Accuracy	Precision	Recall	F1	FPR
Isolation Forest	95.92%	78.92%	100.00%	88.21%	4.81%
Extra Trees	100.00%	100.00%	100.00%	100.00%	0.00%

First-seed confusion counts: Isolation Forest TP 2,810, FP 747, TN 14,872, FN 0; Extra Trees TP 2,810, FP 0, TN 15,619, FN 0. The three-seed Isolation Forest FPR ranges from 4.59% to 5.05%; one seed exceeds 5%. Full seed metrics and sample standard deviations are retained in the JSON report.

The apparent perfect Extra Trees separation must be read alongside Section 8.11. It is a result for these collection domains, not a validated 100% cryptojacking-detection capability. Both model artifacts explicitly record deployment_eligible = false.

8.11 Collection confounding and deployment boundary

All attack-file observations come from a single host, localhost.localdomain. Normal-file observations come from four other VM hosts: vm1-graph-analytics, vm2-node, vm3-tomcat and vm4-web-server. Their collection dates also differ. Thus labels are confounded with the host and collection period. A model can separate resource distributions associated with these environments without learning a general attack signature.

Removing hostname and timestamp columns does not remove this indirect signal. Memory distributions and CPU states can still reveal collection conditions. Chronological splitting, purging and duplicate removal reduce some leakage risks but do not create an unseen-host attack evaluation. A dataset containing only one attack host cannot support that claim.

Conclusion	**Supported interpretation**
Large-data training	Completed with 94,526 clean unique source observations; partition and audit counts are reproducible.
Classification metrics	Valid for the selected within-host chronological split under publisher file labels.
100% Extra Trees accuracy	Potentially reflects collection-domain separation; not a general detection guarantee.
Live deployment	Not authorized by this experiment. Models use an incompatible external-host schema.
Quarantine and recovery	Not evaluated by the Kaggle experiment; no actuation or recovery timing was measured.
Patent evidence	More training rows and high classifier metrics do not establish novelty or inventive step.

The next collection must contain benign and attack workloads on every host, reserve genuinely unseen hosts and dates, and include benign CPU-intensive jobs. The live seven-feature process detector requires its own compatible telemetry and validation. The dashboard and the bracketed recovery API remain separate from this offline study.

8.12 Output evidence - SS7

Kaggle training output, split counts and limitations

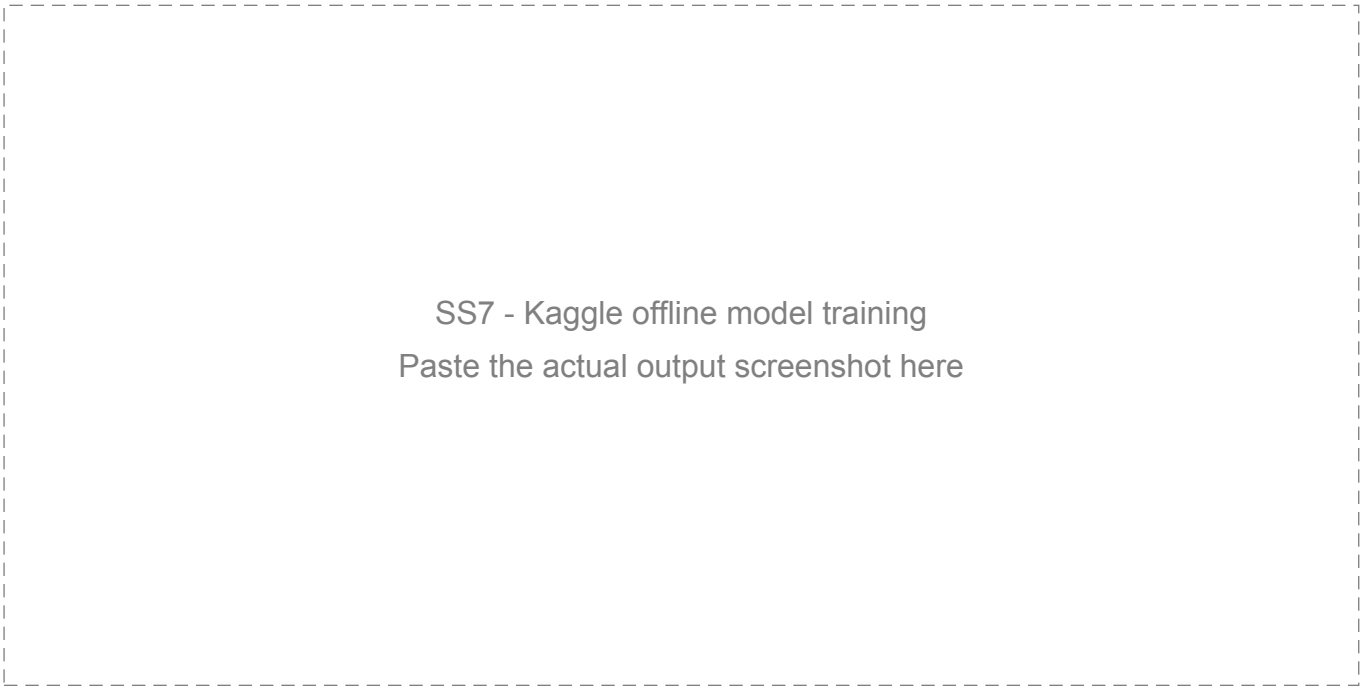


Figure 8. Offline Kaggle model-fit results with dataset size, errors and collection-domain limitation visible.

What screenshot to paste

Capture the command and raw training.log metric lines for both models. Include evaluation.json partition counts and deployment_eligible = false in a second terminal block. Keep a visible note: host/date confounding; offline host telemetry only. Do not show the Extra Trees score without that qualification.

```
.venv/bin/python -m arq.evaluation.kaggle_cryptojacking \
  --archive data/kaggle-cryptojacking/dataset.zip \
  --output output/validation/kaggle-cryptojacking --repeats 3
```

Use the retained results or rerun from the supplied archive and show actual new results. The training package includes the raw archive, models, logs, predictions and split manifest. See docs/KAGGLE_TRAINING.md for the reproduction command and full caveats.

8.13 Live validation plan, ablations and future work

Study	Required evidence / success criterion
Independent workload collection	Multiple days, machines and workload families; keep collection runs and evaluation partitions disjoint.
Controlled recovery experiment	Run a protected service with a restricted owned worker; capture actual duty, CPU time, offered rate and per-request timestamps.
Baselines	Legacy score-driven release, fixed restriction and bracketed release under identical workloads and randomized trial order.
Ablations	Remove normalization, one control window, load stability and drift checks separately; measure premature release and delayed recovery.
Primary outcomes	SLO violation rate, incorrect release, recovery delay, useful throughput and overhead; predefine event labels.
Uncertainty	Repeat independent workload trials; report sample counts, distributions and confidence intervals where justified.
Portability	Execute native telemetry and owned-child cleanup checks on supported Linux and macOS hosts.
Deployment gate	Use sufficient held-out normal/anomalous examples and per-scenario results; retain shadow mode when criteria fail.

Additional work includes an independent watchdog, serialized actuation, measured reapplication timing, multicore accounting, stronger identity handling and an end-to-end integrated recovery path. These are proposed studies and engineering tasks, not completed results.

The current contribution is a reproducible proof of concept and explicit decision contract. It does not establish global optimality. Thresholds trade false release against unnecessary restriction; a claim that this policy is optimal would require a defined objective, constraints and evidence beyond the present tests.

9. What aspect(s) of the invention need(s) protection?

The candidate subject matter is verification of a CPU-restriction relaxation through a particular bracketed observation protocol and acceptance contract. The following draft claim skeleton requires professional novelty, inventive-step and eligibility review. No patent filing or grant is claimed.

1. A computer-implemented method of verifying a proposed relaxation of an existing CPU execution restriction for an identified process incarnation, comprising reserving a nominal additional execution allowance; collecting restricted, relaxed and restricted observation windows; returning to the restricted allocation before a final decision; evaluating observation validity, offered-load stability, restricted-control latency drift, protected-service latency and relaxed-grant-normalized CPU consumption; and applying the proposed allocation only after the required complete window groups satisfy the configured conditions.

2. The method of candidate claim 1, wherein normalized consumption is CPU seconds consumed in the relaxed window divided by the product of its duration and reported effective duty allocation.

3. The method of candidate claim 1, wherein protected latency satisfies both an absolute SLO and a permitted increase over the median p95 of the two restricted control windows.

4. The method of candidate claim 1, wherein changed process incarnation, invalid measurements, changing offered load or excessive control drift prevents a positive restoration decision.

5. The method of candidate claim 1, wherein the original restriction is reapplied on exception and before a successful final relaxation, with a further restoration attempt if that final actuation fails.

6. A processor and memory configured to execute candidate claim 1 and retain an observation-phase trace and decision reason.

10. What is Technology readiness level of your invention? (Tick the appropriate TRL)

Research			Development			Deployment		
TRL 1	**TRL 2**	**TRL 3**	**TRL 4**	**TRL 5**	**TRL 6**	**TRL 7**	**TRL 8**	**TRL 9**
Basic Principles observed	Technology concept formulated	Experimental proof of concept	Technology validated in a lab	Technology validated in a relevant environment (industrially relevant in case of key enabling technologies)	Technology demonstrated in a relevant environment (industrially relevant in case of key enabling technologies)	System prototype demonstration in an operational environment	System complete and qualified	Actual system proven in an operational environment (competitive manufacturing in case of key enabling technologies, or in space)
		✓						

TRL selection: TRL 3 - experimental proof of concept. The implemented bracketed runner, regression tests and synthetic evaluation support this level. Operational deployment, industrial qualification and broader workload validation are not established.

Disclosure, inventorship and institutional review

The project is publicly available at the repository below. The upload containing the research implementation and earlier combined disclosure was verified at commit 5fd5a30 on 30 October 2026. Earlier public availability and any filing dates require separate evidence. Publication is material to any patent review; changing visibility later does not erase it.

Confirm actual human inventorship, AI assistance, applicant details and institutional ownership with VIT and an Indian patent agent. The document-control numbers on the first page reproduce the supplied template for formatting; confirm the current authorized form before submission.

https://github.com/rithika20252024/Adaptive-Resource-Quarantine

Research references [1-5]

[1] Ibidunmoye, Hernandez-Rodriguez and Elmroth (2015). Performance Anomaly Detection and Bottleneck Identification. ACM Computing Surveys.

https://www.diva-portal.org/smash/get/diva2%3A839537/FULLTEXT01.pdf

[2] Liu, Ting and Zhou (2008). Isolation Forest. ICDM.

https://www.lamda.nju.edu.cn/publication/icdm08b.pdf

[3] Hochreiter and Schmidhuber (1997). Long Short-Term Memory. Neural Computation.

https://doi.org/10.1162/neco.1997.9.8.1735

[4] Malhotra et al. (2016). LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection.

https://arxiv.org/abs/1607.00148

[5] Lo et al. (2016). Improving Resource Efficiency at Scale with Heracles. ACM TOCS.

https://research.google/pubs/improving-resource-efficiency-at-scale-with-heracles/

The bibliography contains 25 distinct research papers, including author-hosted preprints explicitly identified as such. Patents and software below are additional sources and are not counted toward the 25-paper minimum.

Research references [6-10]

[6] Fried et al. (2020). Caladan: Mitigating Interference at Microsecond Timescales. OSDI.

https://www.usenix.org/conference/osdi20/presentation/fried

[7] Chen, Delimitrou and Martinez (2019). PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. ASPLOS.

https://www.csl.cornell.edu/~martinez/doc/asplos19.pdf

[8] Ousterhout et al. (2019). Shenango: Achieving High CPU Efficiency for Latency-sensitive Datacenter Workloads. NSDI.

https://www.usenix.org/conference/nsdi19/presentation/ousterhout

[9] Zhang et al. (2013). CPI2: CPU performance isolation for shared compute clusters. EuroSys.

https://research.google/pubs/cpi2-cpu-performance-isolation-for-shared-compute-clusters/

[10] Delimitrou and Kozyrakis (2014). Quasar: Resource-Efficient and QoS-Aware Cluster Management. ASPLOS.

https://csl.stanford.edu/~christos/publications/2014.quasar.asplos.pdf

The bibliography contains 25 distinct research papers, including author-hosted preprints explicitly identified as such. Patents and software below are additional sources and are not counted toward the 25-paper minimum.

Research references [11-15]

[11] Delimitrou and Kozyrakis (2013). Paragon: QoS-aware scheduling for heterogeneous datacenters. ASPLOS.

https://mast.stanford.edu/pubs/paragon_qos_aware_scheduling_for_heterogeneous_datacenters/

[12] Mars et al. (2011). Bubble-Up: Increasing Utilization In Modern Warehouse Scale Computers Via Sensible Co-Locations. MICRO.

https://research.google/pubs/bubble-up-increasing-utilization-in-modern-warehouse-scale-computers-via-sensible-co-locations/

[13] Banga, Druschel and Mogul (1999). Resource Containers: A New Facility for Resource Management in Server Systems. OSDI.

https://www.usenix.org/conference/osdi-99/presentation/resource-containers-new-facility-resource-management-server-systems

[14] Hindman et al. (2011). Mesos: A Platform for Fine-Grained Resource Sharing in the Data Center. NSDI.

https://www.usenix.org/conference/nsdi11/mesos-platform-fine-grained-resource-sharing-data-center

[15] Verma et al. (2015). Large-scale cluster management at Google with Borg. EuroSys.

https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

The bibliography contains 25 distinct research papers, including author-hosted preprints explicitly identified as such. Patents and software below are additional sources and are not counted toward the 25-paper minimum.

Research references [16-20]

[16] Schwarzkopf et al. (2013). Omega: flexible, scalable schedulers for large compute clusters. EuroSys.

https://research.google/pubs/omega-flexible-scalable-schedulers-for-large-compute-clusters/

[17] Ghodsi et al. (2011). Dominant Resource Fairness: Fair Allocation of Multiple Resource Types. NSDI.

https://www.usenix.org/conference/nsdi11/dominant-resource-fairness-fair-allocation-multiple-resource-types

[18] Rzadca et al. (2020). Autopilot: Workload Autoscaling at Google Scale. EuroSys.

https://research.google/pubs/autopilot-workload-autoscaling-at-google-scale/

[19] Kulkarni et al. (2019). Pliant: Leveraging Approximation to Improve Datacenter Resource Efficiency. HPCA.

https://www.csl.cornell.edu/~nsk49/publication/pliant_hpca/

[20] Belay et al. (2012). Dune: Safe User-level Access to Privileged CPU Features. OSDI.

https://www.usenix.org/conference/osdi12/technical-sessions/presentation/belay

The bibliography contains 25 distinct research papers, including author-hosted preprints explicitly identified as such. Patents and software below are additional sources and are not counted toward the 25-paper minimum.

Research references [21-25]

[21] Kaffes et al. (2019). Shinjuku: Preemptive Scheduling for Microsecond-scale Tail Latency. NSDI.

https://www.usenix.org/conference/nsdi19/presentation/kaffes

[22] Cho et al. (2020). Overload Control for Microsecond-scale RPCs with Breakwater. OSDI.

https://www.usenix.org/conference/osdi20/presentation/cho

[23] Qiu et al. (2020). FIRM: An Intelligent Fine-grained Resource Management Framework for SLO-Oriented Microservices. OSDI.

https://www.usenix.org/conference/osdi20/presentation/qiu

[24] Zhang et al. (2021). Sinan: Data-Driven, QoS-Aware Cluster Management for Microservices. arXiv:2105.13424.

https://arxiv.org/abs/2105.13424

[25] Gan et al. (2021). Sage: Leveraging ML to Diagnose Unpredictable Performance in Cloud Microservices. arXiv:2112.06263.

https://arxiv.org/abs/2112.06263

The bibliography contains 25 distinct research papers, including author-hosted preprints explicitly identified as such. Patents and software below are additional sources and are not counted toward the 25-paper minimum.

Additional references and screenshot checklist

Suspicious-thread quarantine, US7530072B1

https://patents.google.com/patent/US7530072B1/en

Isolation of misbehaving processes, US9280662B2

https://patents.google.com/patent/US9280662B2/en

Heracles

https://research.google/pubs/improving-resource-efficiency-at-scale-with-heracles/

PARTIES

https://www.csl.cornell.edu/~martinez/doc/asplos19.pdf

CPI2

https://research.google/pubs/cpi2-cpu-performance-isolation-for-shared-compute-clusters/

Bubble-Up

https://research.google/pubs/bubble-up-increasing-utilization-in-modern-warehouse-scale-computers-via-sensible-co-locations/

Dynamic resources, US20230128059A1

https://patents.google.com/patent/US20230128059A1/en

cpulimit

https://github.com/opsengine/cpulimit

Kaggle cryptojacking performance dataset (Keshani Jayasinghe; CC0)

https://www.kaggle.com/datasets/keshanijayasinghe/cryptojacking-attack-timeseries-dataset

Screenshot checklist: SS1 dashboard; SS2 full tests; SS3 labelled evaluation; SS4 assistance gate rejection; SS5 repeated live trial summary; SS6 synthetic recovery comparison. SS7 Kaggle offline training with collection-domain caveat. All seven spaces and paste instructions appear within Section 8.